

Velo

Manual técnico do projeto

E-commerce completo em Java, Spring Boot e React

Feito por Erik L. Pimenta

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral 4

- Para que serve 4
- O que a Velo faz 4
- Tecnologias 5
- Identidade visual 5

Como rodar 6

- Primeira execução 6
- Gerar o arquivo .jar 6
- Desenvolvendo a interface 6
- Estrutura de pastas 7

Acesso e sessão 8

- Contas de demonstração 8
- Regras da sessão 9

Guia da loja 10

- Vitrine 10
- Produto 10
- Carrinho 11
- Entrega e pagamento 11
- Pedidos 12
- Minha conta 13
- No celular 14

Painel administrativo 15

- Perfis e permissões 15
- Visão geral 15
- Pedidos 16
- Produtos e estoque 17
- Categorias, cupons e usuários 17

Pedidos e pagamentos 18

Ciclo do pedido 18

Regras de valor e estoque 19

Pagamento simulado 19

Stripe 20

Arquitetura 21

Backend por pacote 21

Interface por pasta 22

Decisões principais 22

Erros 22

API 23

Banco de dados 25

Pareando com outro banco 25

Configuração, testes e automação 26

Variáveis de ambiente 26

Testes 26

Integração contínua 26

Commits automáticos 26

Solução de problemas 27

Capítulo 1

Visão geral

A Velo é uma loja virtual completa e de código aberto. Ela cobre o caminho inteiro de uma venda on-line: o cliente encontra o produto, monta o carrinho, paga e acompanha a entrega, enquanto a equipe cuida de pedidos, estoque, cupons e usuários num painel próprio.

Para que serve

O projeto foi feito para ser usado, estudado e adaptado. Qualquer pessoa pode baixar o código, rodar com um comando, entrar com as contas de demonstração e ver uma loja funcionando, com dados de exemplo, pagamentos simulados e um painel administrativo completo. Para colocar uma loja real no ar basta trocar o banco, as senhas e, se quiser cobrar de verdade, ligar a Stripe.

O que a Velo faz

Para quem compra

- Vitrine com busca, filtro por categoria, ordenação e paginação.
- Página de produto com estoque em tempo real, preço anterior riscado e parcelamento.
- Carrinho em gaveta lateral e em página completa, com barra de frete grátis e cupom de desconto.
- Checkout em uma tela: endereço salvo ou novo, cartão em até 10 vezes, PIX ou boleto.
- Acompanhamento do pedido com linha do tempo, rastreio, nova tentativa de pagamento e cancelamento.
- Conta com dados pessoais, endereços e troca de senha.

Para quem vende

- Visão geral com faturamento, ticket médio, vendas por dia, mais vendidos, estoque baixo e pedidos recentes.
- Pedidos com busca, filtros, mudança de situação, código de rastreio, cancelamento e estorno.
- Produtos com cadastro completo, movimentação de estoque (entrada, saída e contagem) e histórico.
- Categorias, cupons (percentual ou valor fixo, com mínimo, limite de usos e validade) e usuários com três perfis.

Por baixo

- Estoque reservado com trava no banco no momento da compra e devolvido sozinho ao cancelar ou estornar.
- Pedidos não pagos em 60 minutos são cancelados automaticamente.
- Sessão com limite de 20 minutos parado e 30 minutos no total, conferida pelo servidor a cada chamada.
- Um único modelo de banco, versionado com Flyway, que roda em seis bancos diferentes.

Tecnologias

Camada	Tecnologia	Papel
Linguagem	Java 21	Toda a regra de negócio e a API
API	Spring Boot 4.0.8 (Web MVC, Security, Validation, Data JPA)	Rotas REST, segurança, validação e acesso a dados
Banco	Hibernate 7 + Flyway	Mapeamento das tabelas e criação do esquema
JSON	Jackson 3	Conversão de dados e leitura do webhook da Stripe
Interface	React 19 + TypeScript 5.9	Telas da loja e do painel
Build da interface	Vite 7	Servidor de desenvolvimento e pacote final
Animações	Motion 12	Gaveta do carrinho, modais e avisos, sempre em resposta a uma ação
Fonte	Public Sans	Tipografia de toda a interface
Scripts	Python 3	Gerador do esquema do banco e das ilustrações
Testes	JUnit 6 + Spring Boot Test	Sessão, compra, permissões e regras
Bancos	H2, PostgreSQL, MySQL, MariaDB, SQL Server, Oracle	Escolha de quem instala

Identidade visual

A interface é sóbria e minimalista, feita para quem usa a loja e o painel todos os dias. O fundo é um cinza bem claro, as superfícies são brancas com bordas finas, e um único tom de destaque, o verde-pinho, aparece nos gráficos, na seleção e no foco do teclado.

#F6F7F7
Fundo das páginas

#FFFFFF
Cartões, tabelas e formulários

#E2E5E6
Bordas e divisórias

#5F666B
Textos de apoio

#15181B
Texto e botões principais

#1F5C4C
Destaque: gráficos, seleção e foco

As cores de estado aparecem só em etiquetas, avisos e variações de valor: sucesso `#1F6B45`, aviso `#8A5A12` e erro `#A03A32`.

A fonte é a Public Sans, com títulos de página de 24 px e peso 600, texto de 14,5 px e números tabulares em preços e totais. Os cantos têm de 4 a 8 px, os botões são retangulares e as situações dos pedidos aparecem em etiquetas discretas. As animações acontecem só em resposta a uma ação (abrir o carrinho, um modal ou um aviso); trocar de página não tem efeito de entrada nem cascata. As ilustrações dos produtos usam um fundo cinza neutro único e são geradas por `scripts/imagens/gerar_imagens.py`.

A marca é uma sacola com linhas de velocidade, em `#15181B`. Ela aparece desenhada em vetor no cabeçalho, no painel, na tela de entrada e no ícone da aba; as versões em imagem ficam em `docs/images/logo.png` (fundo preto) e `docs/images/logo-transparente.png` (sem fundo).

Capítulo 2

Como rodar

Só é preciso ter o Java 21 instalado. O Maven baixa o Node, compila a interface e embute tudo num único servidor.

Primeira execução

Windows

1. Instale o Java 21 (por exemplo, o Temurin 21 da Adoptium) e confirme com `java -version`.
2. Abra a pasta do projeto e dê dois cliques em `iniciar.cmd`.
3. Espere aparecer Started VeloApplication na janela. A primeira vez leva alguns minutos para baixar as dependências.
4. Abra **http://localhost:8080** no navegador e entre com uma das contas de demonstração.

Linux ou macOS

```
./iniciar.sh
```

Sem configurar nada, a Velo usa um banco H2 em arquivo, em `backend/data`, já preenchido com os dados de demonstração. Para começar do zero, pare a aplicação e apague essa pasta.

Gerar o arquivo .jar

```
cd backend
./mvnw -B package
java -jar target/velo.jar
```

No Windows, use `mvnw.cmd` no lugar de `./mvnw`. O .jar já contém a interface e pode ser copiado para qualquer servidor com Java 21.

Desenvolvendo a interface

Para mexer nas telas com recarga instantânea, rode a API sem compilar a interface e use o servidor do Vite:

```
cd backend
./mvnw -B spring-boot:run -DskipUi
```

```
cd frontend
npm install
npm run dev
```

Abra **http://localhost:5173**. As chamadas para `/api` são encaminhadas para a porta 8080.

Estrutura de pastas

Pasta	Conteúdo
`backend/`	API Spring Boot, migrações do banco, configuração e testes
`backend/src/main/java/dev/velo/`	Código Java dividido por assunto (veja o capítulo de arquitetura)
`backend/src/main/resources/db/migration/`	Um V1__schema.sql para cada banco suportado
`frontend/`	Interface React + TypeScript
`frontend/public/products/`	Ilustrações dos 24 produtos de demonstração
`scripts/schema/`	Gerador do esquema e da documentação do banco
`scripts/imagens/`	Gerador das ilustrações dos produtos
`docs/`	Guia do banco, diagrama em PDF e imagens da marca
`docker/` e `docker-compose.yml`	Bancos prontos para teste com Docker
`.github/workflows/`	Integração contínua e commits automáticos

Capítulo 3

Acesso e sessão

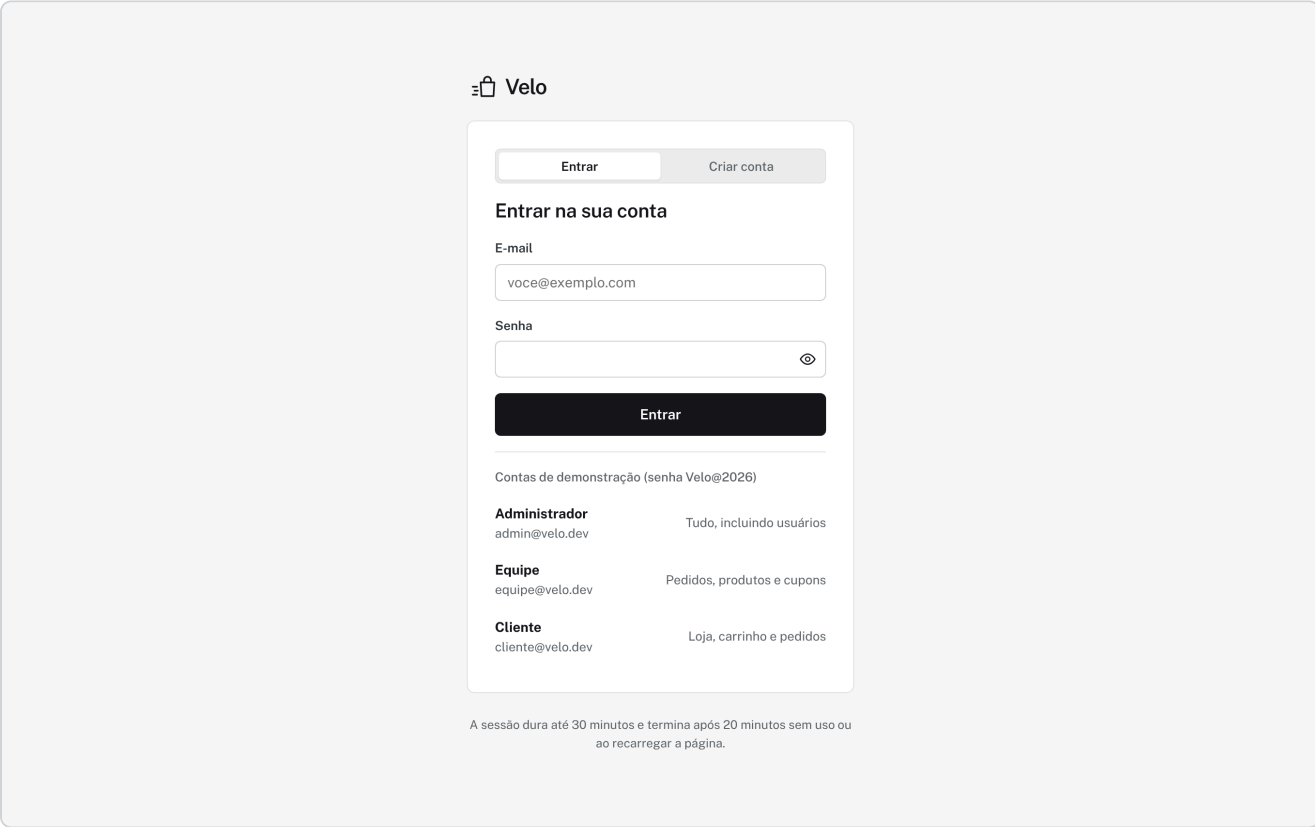
Toda tela e toda rota da API exigem login. A sessão é curta de propósito e é conferida tanto pelo navegador quanto pelo servidor.

Contas de demonstração

Todas usam a senha **Velo@2026**. Elas estão no código de propósito, para qualquer pessoa conseguir entrar logo depois de baixar o projeto. A tela de entrada é um cartão branco centralizado sobre o fundo cinza, com a marca acima, as abas Entrar e Criar conta e o título Entrar na sua conta. As contas de demonstração ficam em lista logo abaixo do formulário: basta clicar numa delas para preencher o e-mail e a senha. Abaixo do cartão, um aviso lembra quanto tempo dura a sessão.

Perfil	E-mail	O que pode fazer
Administrador	admin@velo.dev	Tudo, incluindo criar, editar e bloquear usuários
Equipe	equipe@velo.dev	Painel, pedidos, produtos, categorias e cupons
Cliente	cliente@velo.dev	Loja, carrinho, checkout, pedidos e conta

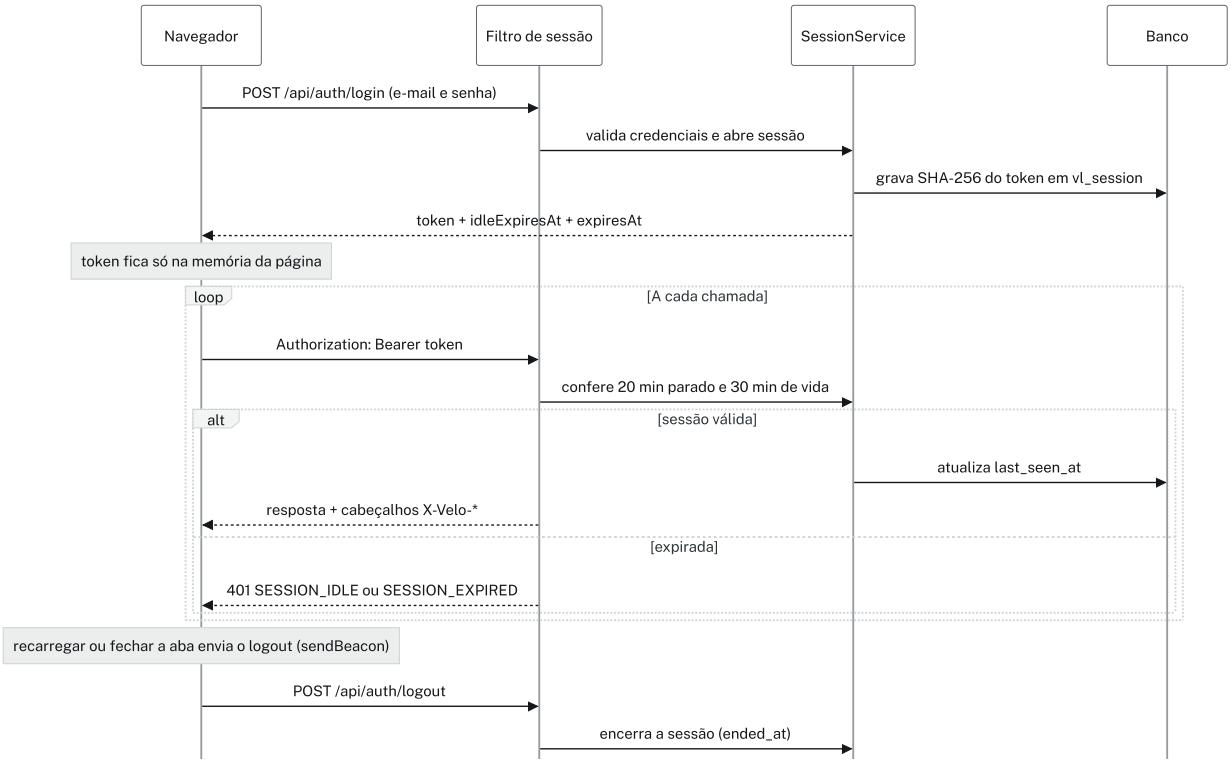
Os dados de demonstração trazem 24 produtos em 6 categorias, 9 clientes fictícios, 64 pedidos de exemplo e os cupons `BEMVINDO10` (10%), `VELO20` (20% acima de R$ 150) e `INVERNO15` (inativo). Para uma loja real, troque essas senhas ou suba com `VELO_SEED=false` num banco vazio.



Tela de entrada com as contas de demonstração

Regras da sessão

Regra	Como funciona
Rotas protegidas	Sem login, qualquer endereço leva à tela de entrada. O painel exige perfil Equipe ou Administrador; a tela de usuários só abre para Administrador.
Recarregou, saiu	O token fica só na memória da página. Recarregar, fechar a aba ou abrir outra aba exige entrar de novo, e a sessão é encerrada no servidor.
20 minutos parado	Sem cliques, teclas ou rolagem por 20 minutos, a sessão termina. No último minuto aparece um aviso com a opção de continuar.
30 minutos no total	Mesmo em uso, a sessão dura no máximo 30 minutos. Um aviso aparece no último minuto.
Conferência no servidor	Os dois limites são verificados pela API a cada chamada. Enquanto a pessoa usa a tela, o navegador avisa o servidor uma vez por minuto.
Bloqueio de senha	Cinco tentativas erradas bloqueiam o login daquele e-mail por 5 minutos.
Senhas e tokens	Senhas ficam só como hash BCrypt; sessões guardam apenas o SHA-256 do token.



Como uma sessão é aberta, conferida e encerrada

Os limites ficam em `application.yml` (`velo.session.idle-minutes` e `velo.session.max-minutes`).

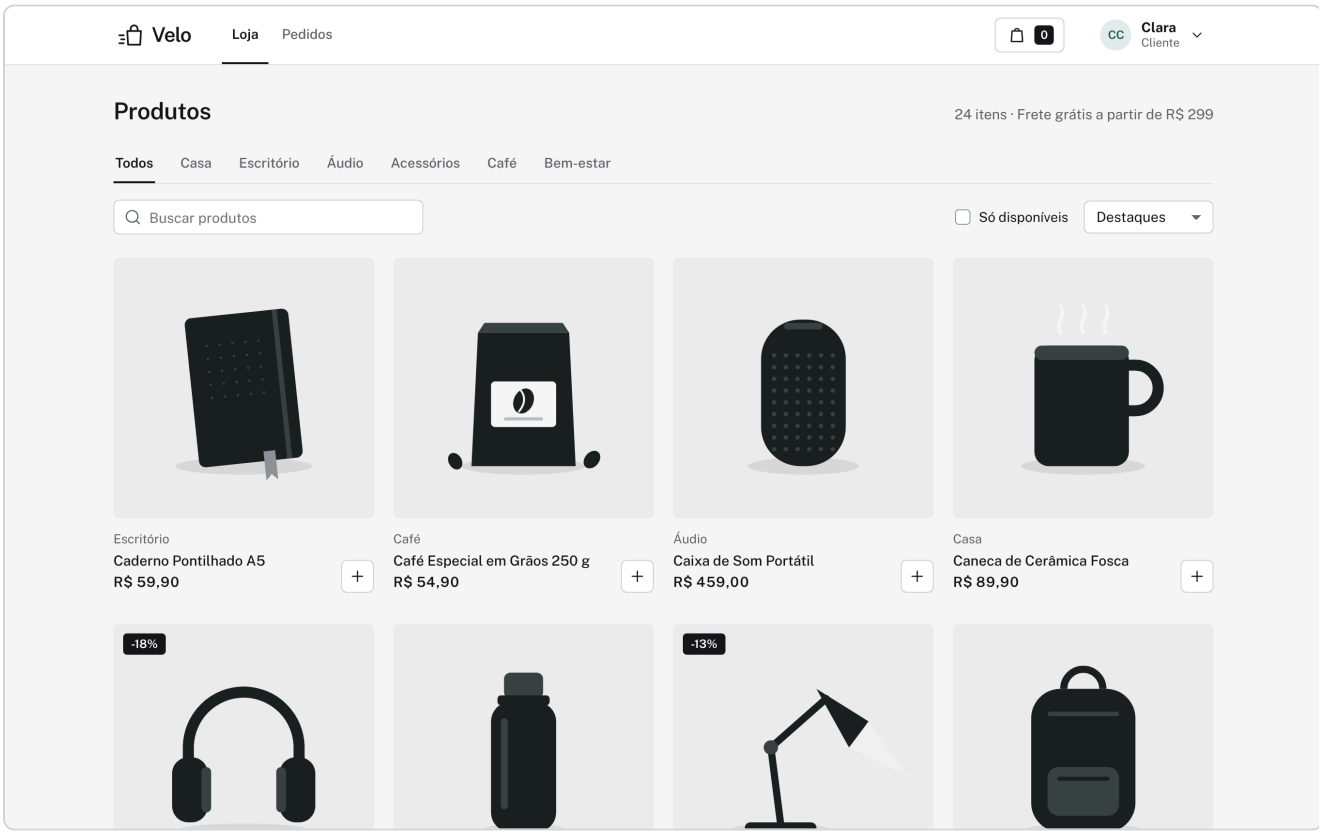
Capítulo 4

Guia da loja

O caminho de quem compra, tela a tela.

Vitrine

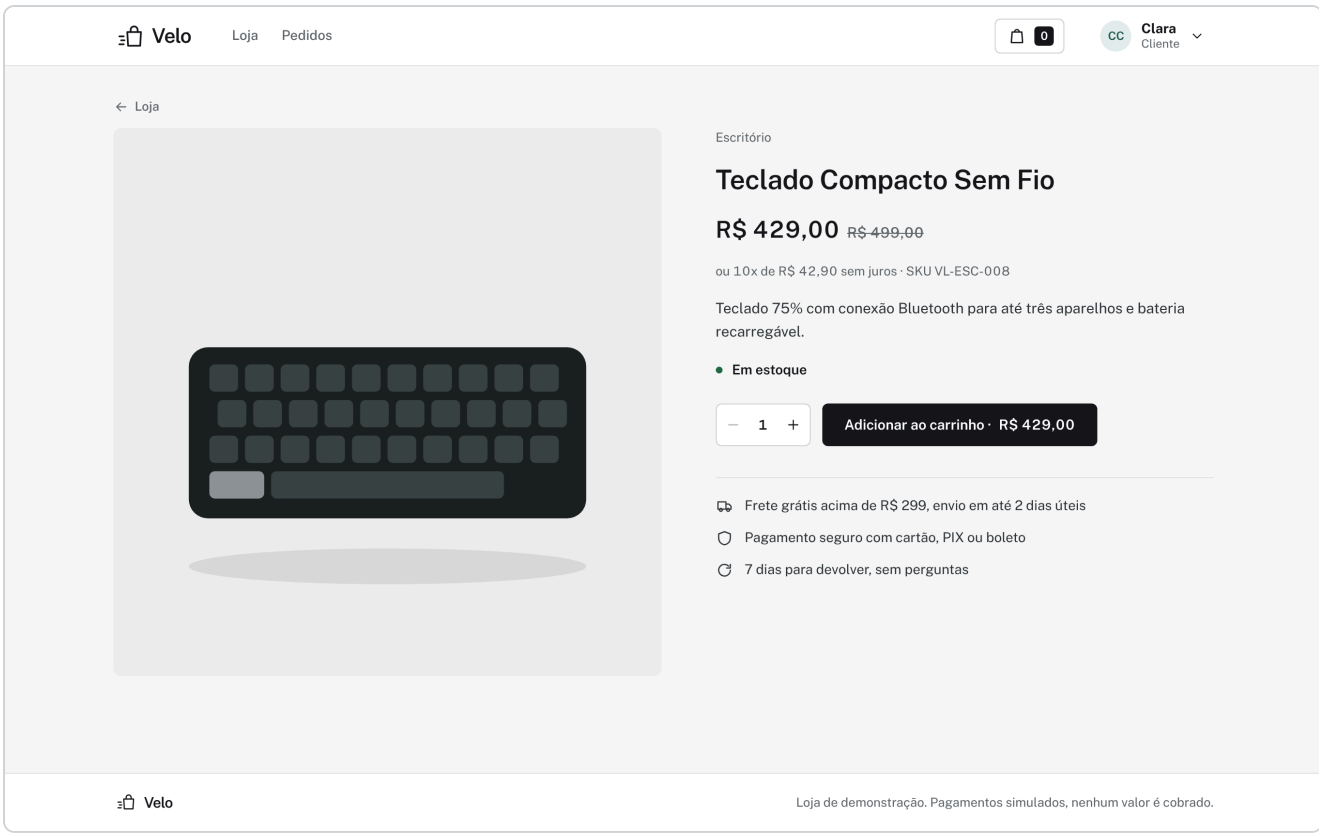
O cabeçalho da loja é branco, com a marca, os links Loja e Pedidos (o ativo fica sublinhado), o botão retangular do carrinho com o contador de itens e o menu do usuário. A página inicial começa pelo título Produtos, com a contagem de itens e o lembrete de frete grátis a partir de R$ 299, e mostra os produtos em grade. As categorias ficam em abas sublinhadas (Todos e uma aba por categoria); a busca, logo abaixo, filtra enquanto se digita, e à direita dá para esconder os esgotados (Só disponíveis) e ordenar por destaque, novidade, preço ou nome. As ilustrações dos produtos têm o mesmo fundo cinza neutro. O botão + adiciona direto ao carrinho.



Vitrine com categorias, busca e ordenação

Produto

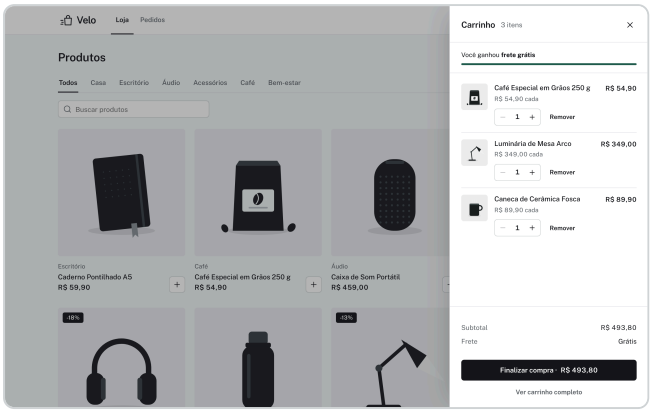
Mostra a descrição, o preço (com o anterior riscado quando há desconto), o parcelamento, o estoque e o seletor de quantidade. Quando restam poucas unidades, a página avisa.



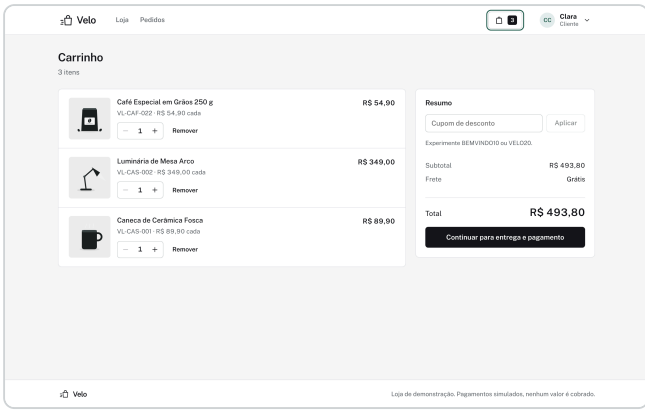
Página do produto

Carrinho

O carrinho abre numa gaveta lateral a partir de qualquer tela, com a barra de frete grátis, e também tem uma página completa com o campo de cupom e o resumo. O frete custa R$ 19,90 e é grátis a partir de R$ 299.



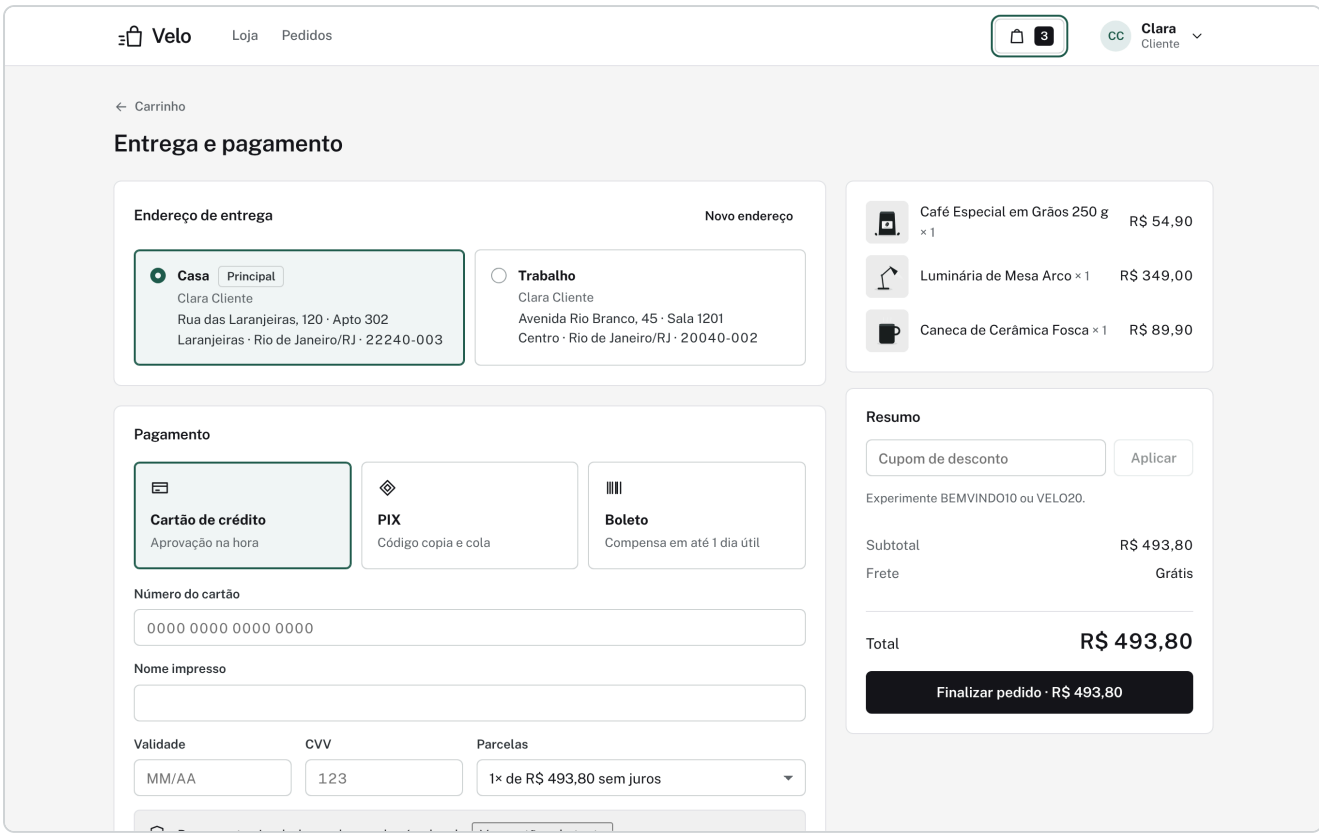
Gaveta do carrinho



Página do carrinho

Entrega e pagamento

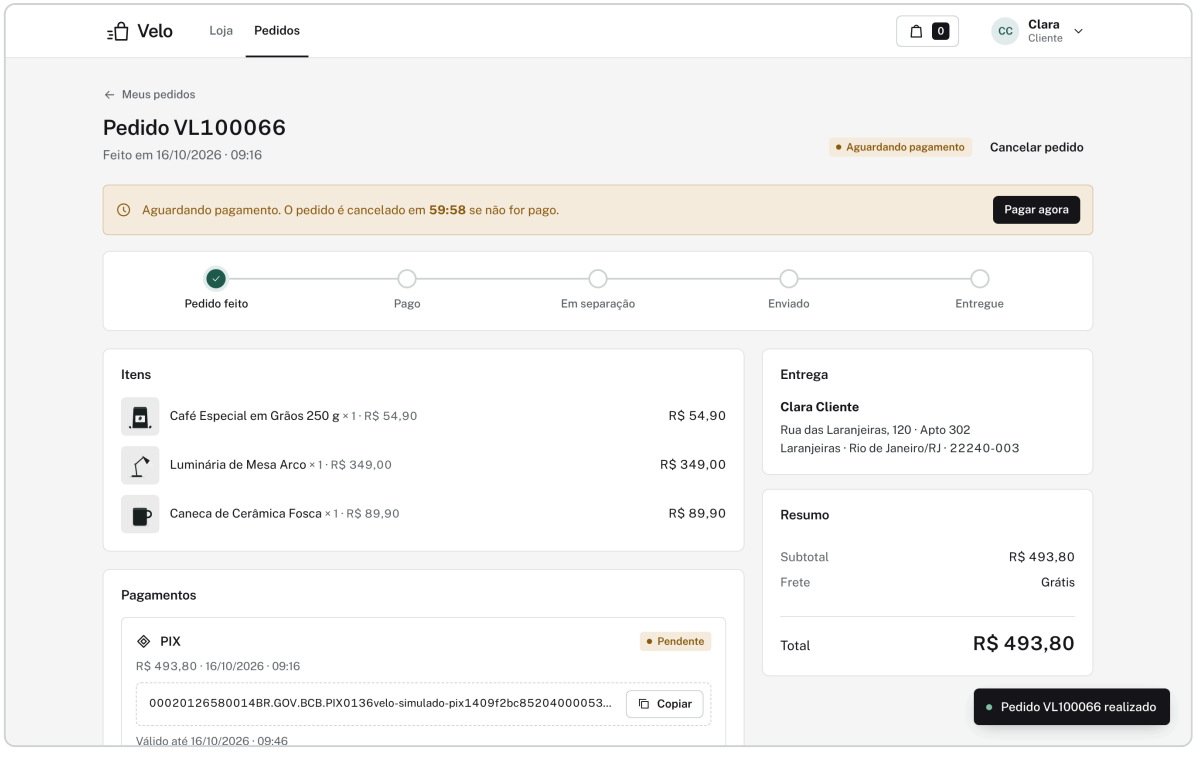
Numa única tela o cliente escolhe um endereço salvo ou cadastra um novo, escolhe a forma de pagamento e finaliza. No modo simulado, o próprio formulário mostra os cartões de teste.



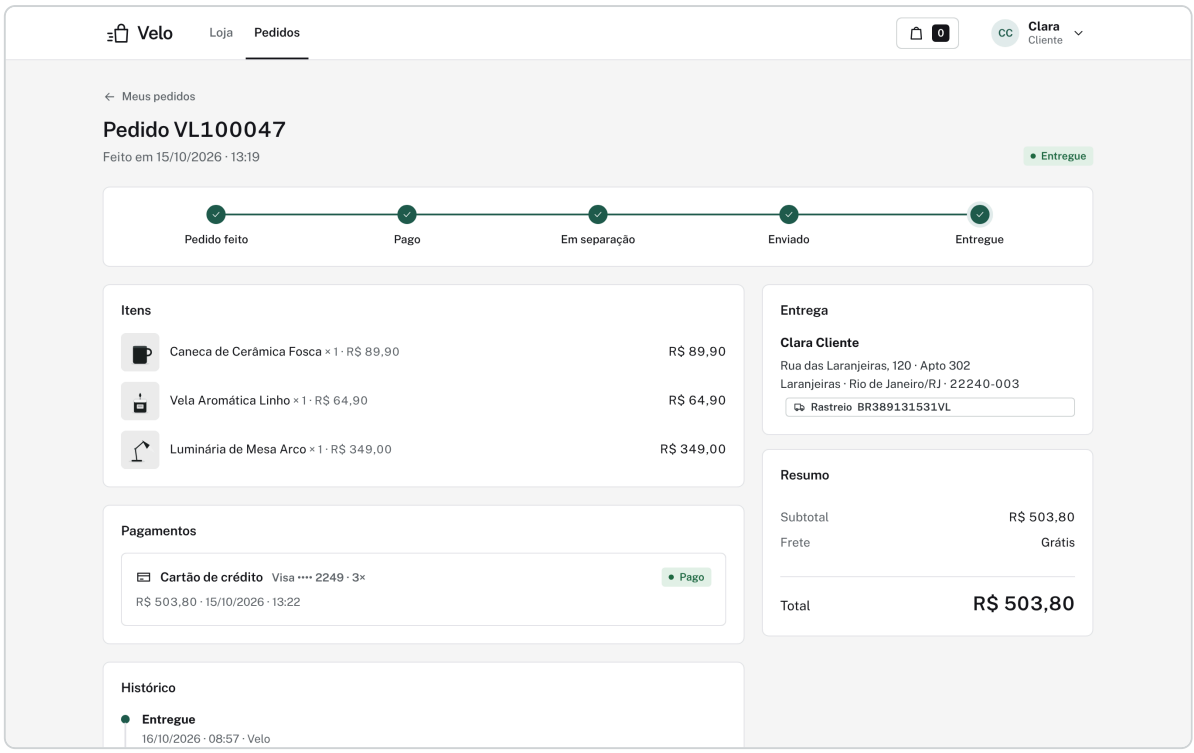
Checkout com endereço, pagamento e resumo

Pedidos

A lista mostra todos os pedidos com filtro por situação. O detalhe traz a barra de andamento, os itens, os pagamentos, o histórico, o endereço e o resumo. Enquanto o pedido aguarda pagamento, um contador mostra quanto falta para o cancelamento automático, e o botão **Pagar agora** permite tentar de novo.



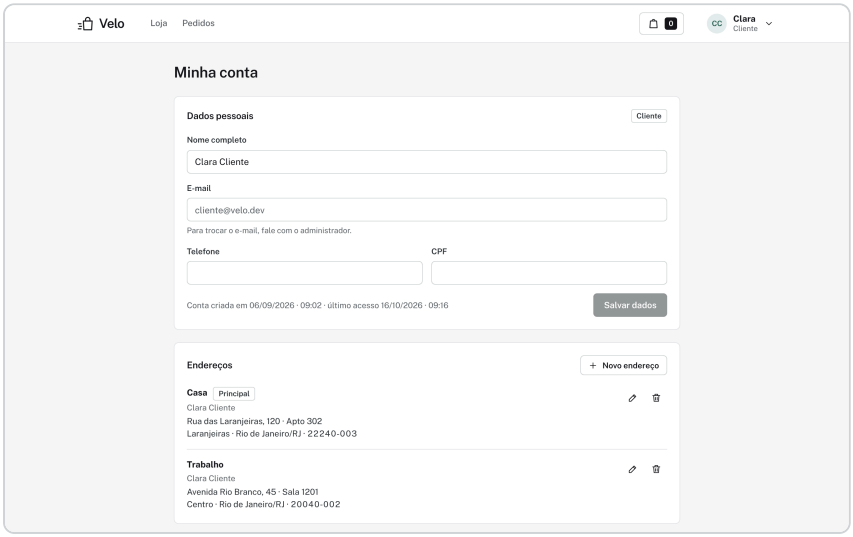
Pedido aguardando pagamento por PIX



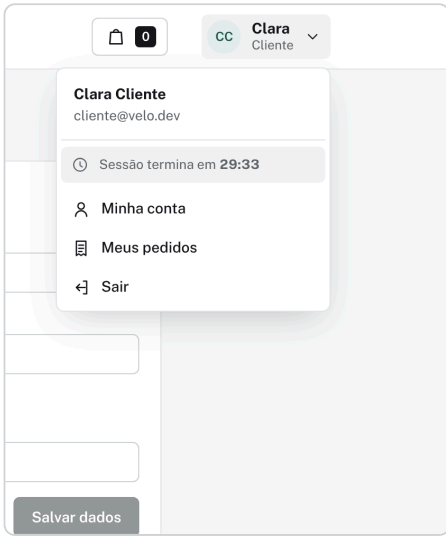
Pedido pago com cartão

Minha conta

Dados pessoais, endereços (com um principal) e troca de senha. O menu do usuário, no canto superior, mostra quanto tempo resta de sessão.



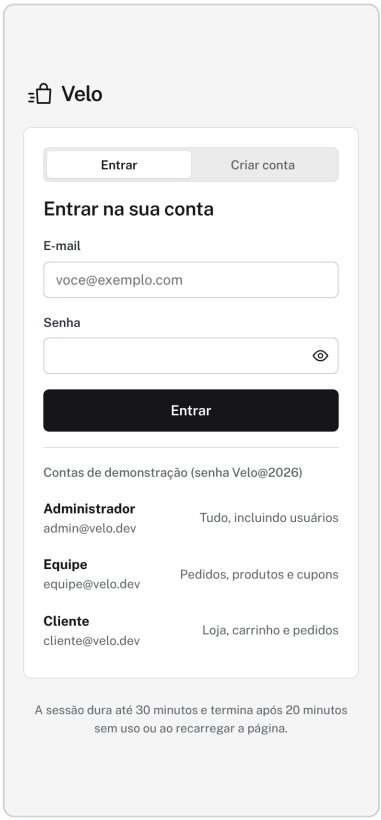
Minha conta



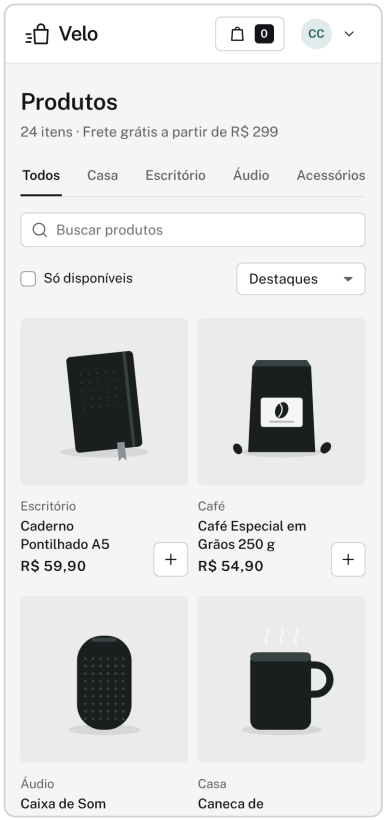
Menu do usuário com o tempo de sessão

No celular

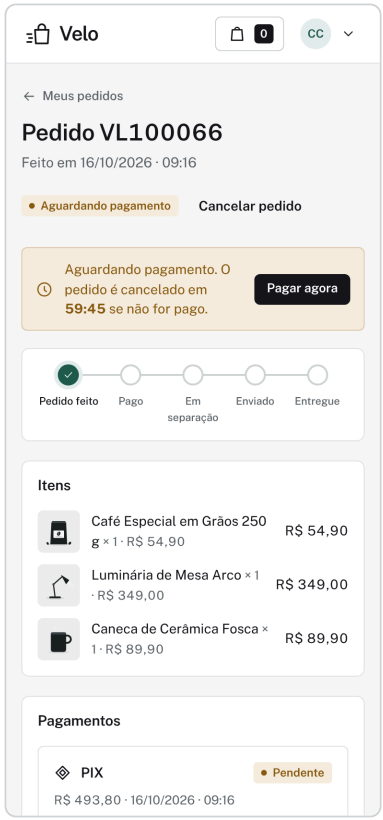
Todas as telas se reorganizam em telas pequenas: a grade vira duas colunas, os formulários ficam em uma coluna e o painel passa a ter a navegação no topo.



Entrada



Vitrine



Pedido

Capítulo 5

Painel administrativo

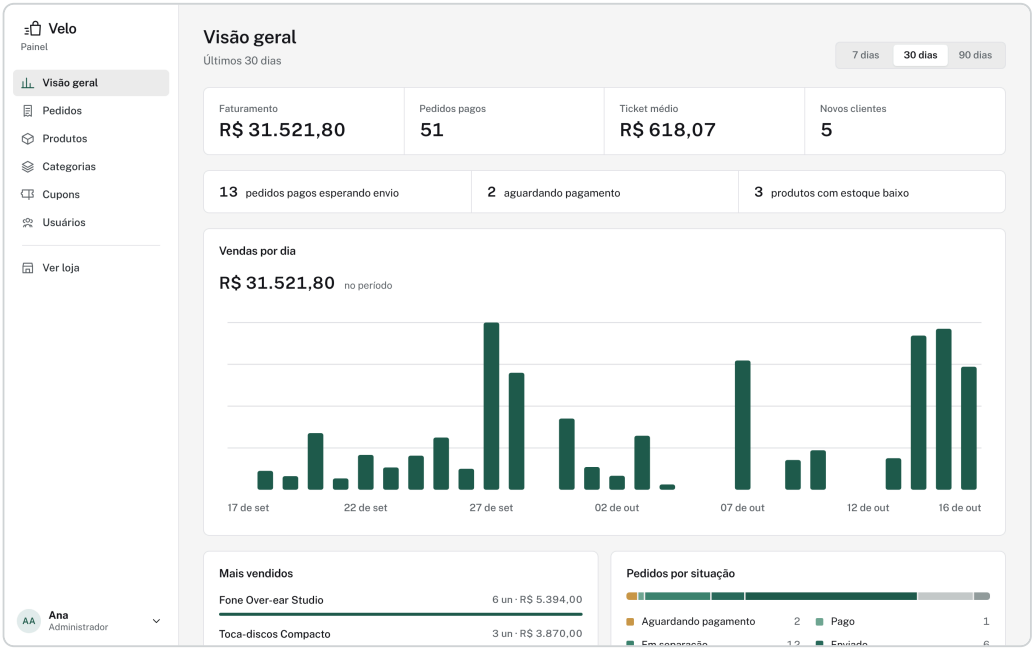
O painel abre para os perfis Equipe e Administrador. A navegação fica numa barra lateral branca, com o item ativo em fundo cinza e ícone verde, e cada tela guarda os filtros no endereço, então dá para compartilhar um link já filtrado.

Perfis e permissões

Área	Cliente	Equipe	Administrador
Loja, carrinho, checkout, pedidos e conta	sim	sim	sim
Visão geral do painel	não	sim	sim
Pedidos: ver, mudar situação, cancelar e estornar	não	sim	sim
Produtos, estoque e categorias	não	sim	sim
Cupons	não	sim	sim
Usuários: criar, editar, bloquear, redefinir senha e encerrar sessões	não	não	sim

Visão geral

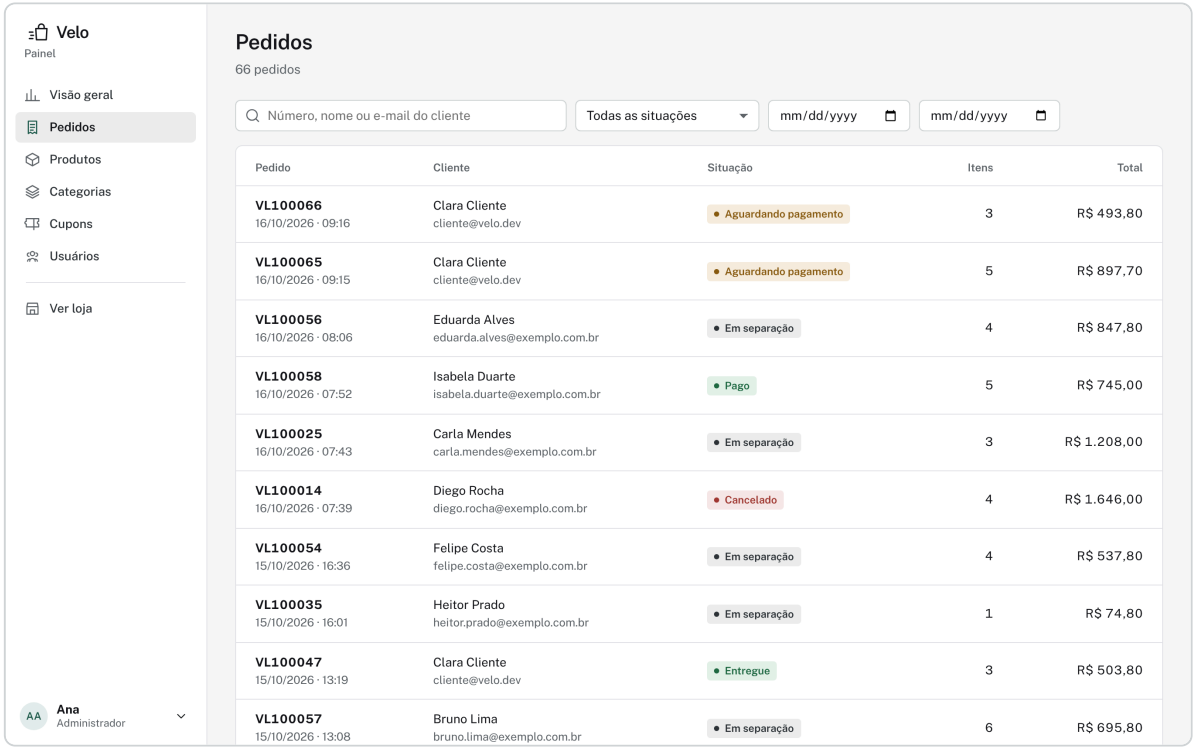
Faturamento, pedidos pagos, ticket médio e novos clientes ficam numa faixa única de indicadores, com troca entre 7, 30 e 90 dias; a variação do faturamento sobre o período anterior aparece em verde quando sobe e em vermelho quando cai. Logo abaixo ficam os atalhos para o que precisa de atenção, o gráfico de vendas por dia em barras verde-pinho (passe o mouse nas barras para ver o valor), os mais vendidos, os pedidos por situação, os pedidos recentes e o estoque baixo. Os avisos de cada ação aparecem no canto inferior direito, e quando a sessão está perto do limite uma faixa âmbar aparece no topo.



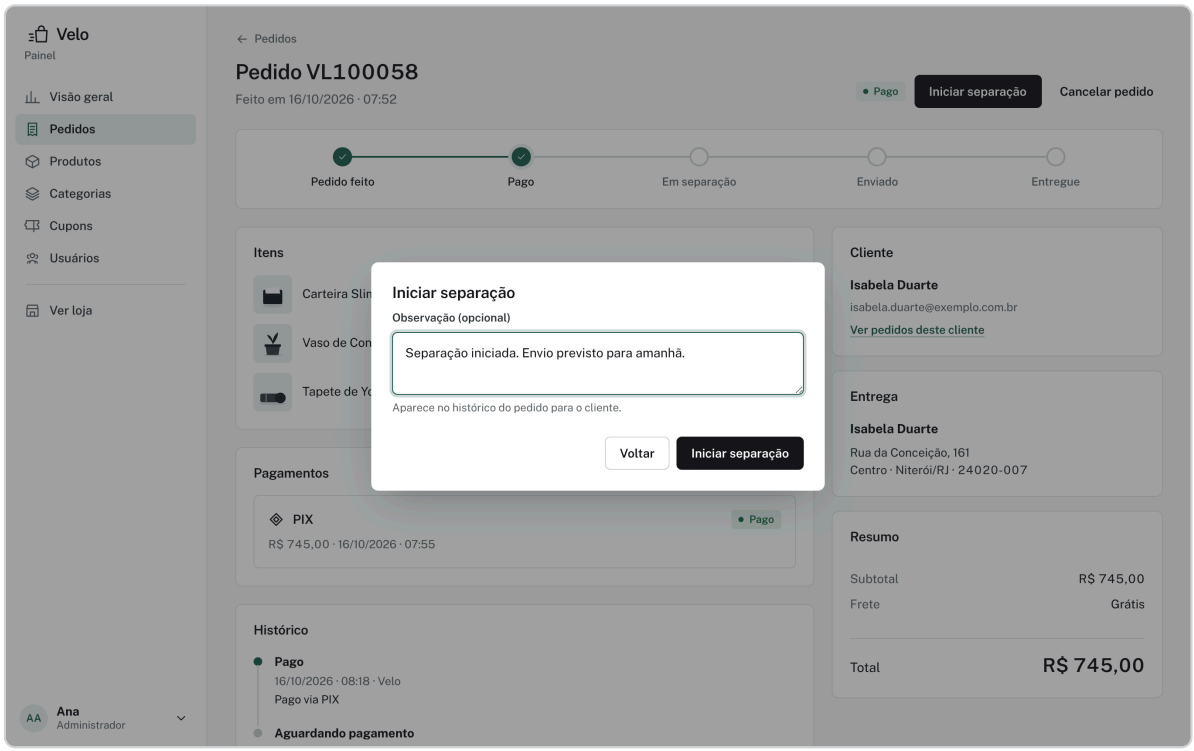
Visão geral do painel

Pedidos

Busca por número, nome ou e-mail do cliente, filtro por situação e por período. No detalhe, os botões no topo mostram só as mudanças permitidas a partir da situação atual. Para marcar como enviado, o código de rastreio é obrigatório; cancelar ou estornar devolve o estoque e, se houve pagamento, o valor.



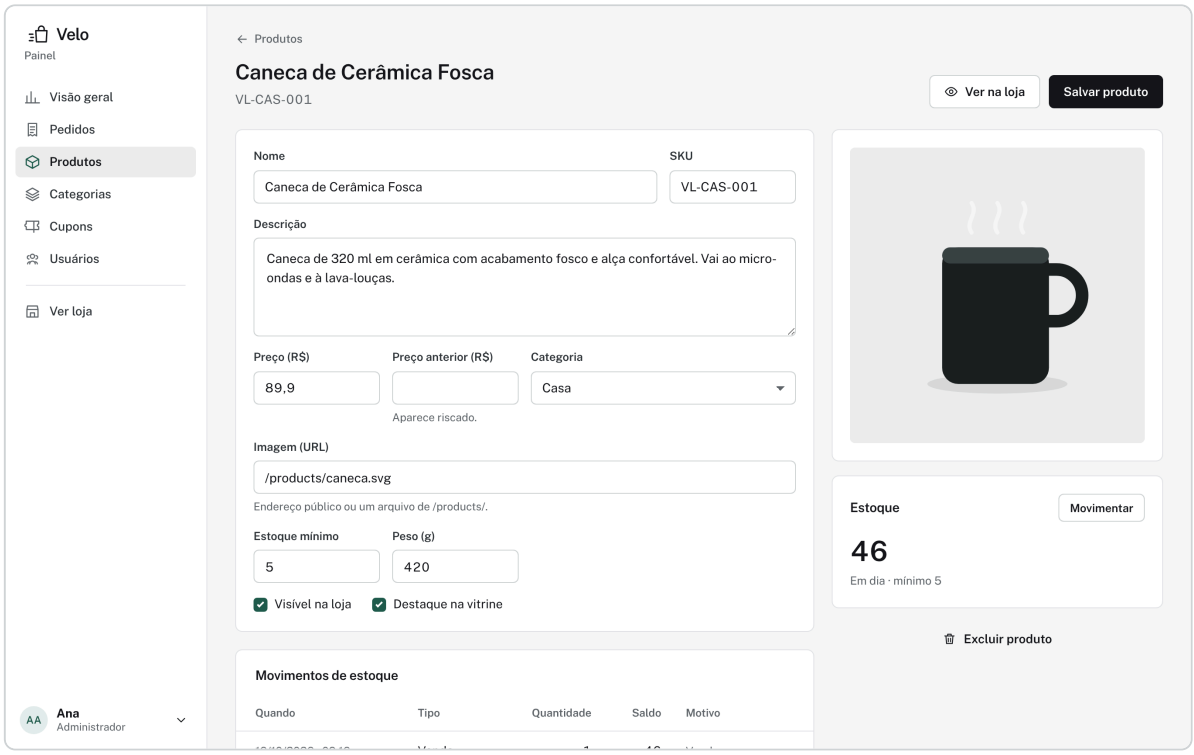
Lista de pedidos com filtros



Mudança de situação com observação para o cliente

Produtos e estoque

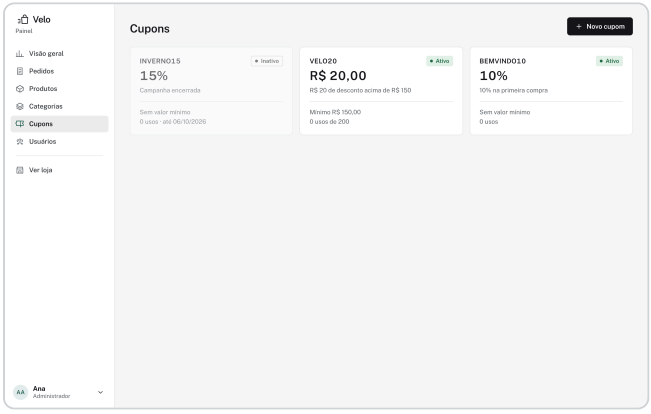
A lista ordena por nome, menor estoque, mais recentes ou maior preço. No cadastro ficam nome, SKU (gerado sozinho se ficar vazio), descrição, preços, categoria, imagem, estoque mínimo, peso, visibilidade e destaque. O estoque só muda por movimentos: **Entrada**, **Saída** ou **Contagem**, e cada um fica registrado com o saldo e o motivo. Um produto que já aparece em pedidos não é apagado, só desativado, para manter o histórico.



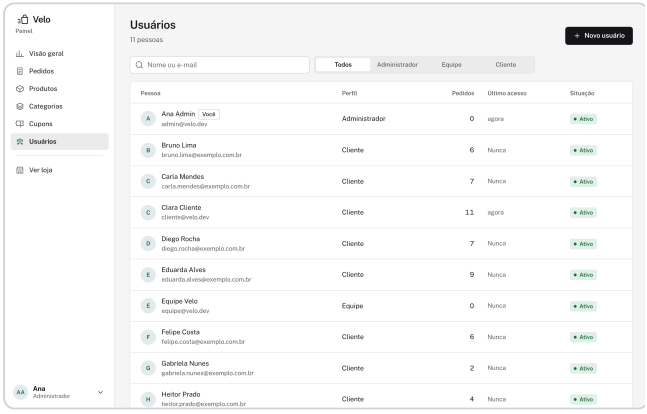
Cadastro do produto com estoque e movimentos

Categorias, cupons e usuários

Categorias têm nome, descrição, ordem de exibição e visibilidade. Cupons podem ser percentuais ou de valor fixo, com pedido mínimo, limite de usos e período de validade. Na tela de usuários, o administrador cria contas, muda perfis, bloqueia o acesso, redefine senhas e encerra sessões abertas. A Velo impede que o último administrador ativo seja rebaixado ou bloqueado, e ninguém consegue tirar o próprio acesso de administrador.



Cupons



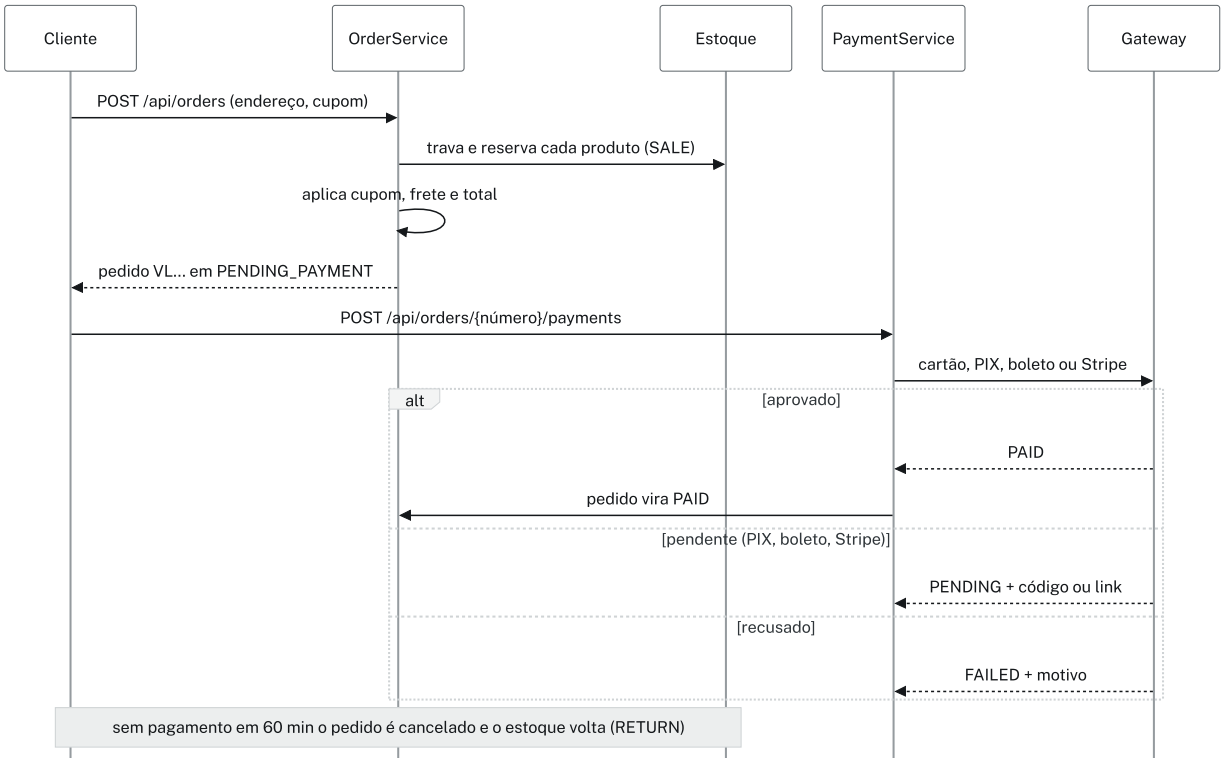
Usuários

Capítulo 6

Pedidos e pagamentos

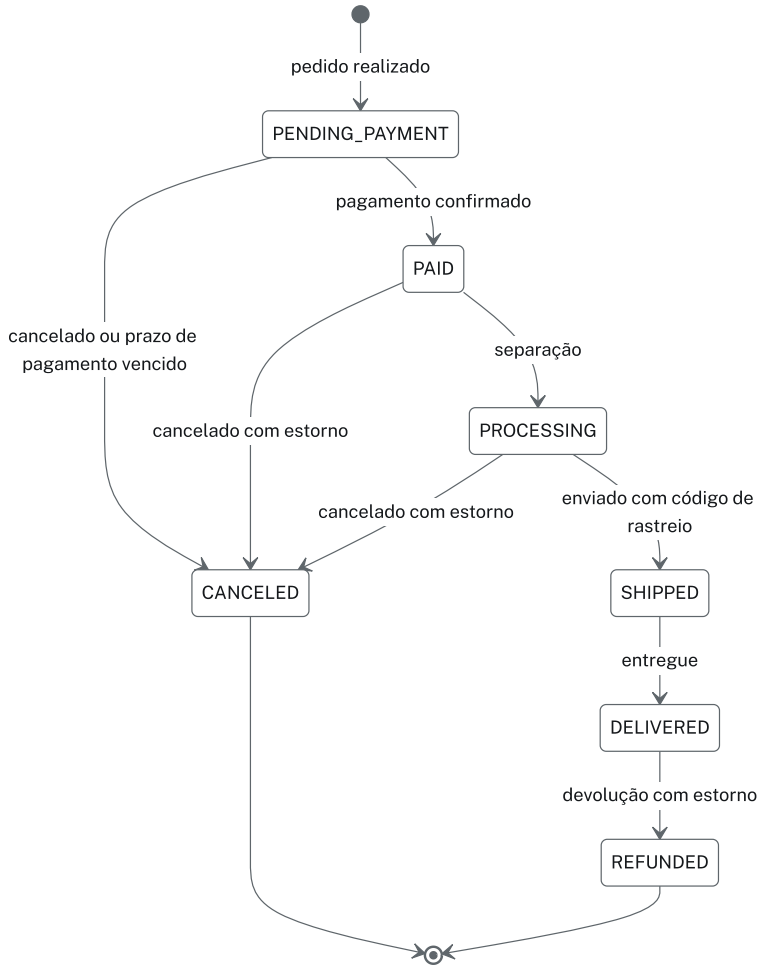
Como um pedido nasce, é pago e termina, e como cada forma de pagamento funciona.

Ciclo do pedido



Da finalização ao pagamento

Situação	Significado	Próximos passos possíveis
`PENDING_PAYMENT`	Aguardando pagamento	PAID, CANCELED
`PAID`	Pago	PROCESSING, CANCELED (com estorno)
`PROCESSING`	Em separação	SHIPPED (exige rastreio), CANCELED
`SHIPPED`	Enviado	DELIVERED
`DELIVERED`	Entregue	REFUNDED (devolução com estorno)
`CANCELED`	Cancelado	Fim
`REFUNDED`	Estornado	Fim



Fluxo de situações do pedido

Regras de valor e estoque

- O estoque de cada item é reservado com trava no banco na finalização (movimento **SALE**) e volta ao cancelar ou estornar (**RETURN**).
- Pedidos sem pagamento em 60 minutos são cancelados por uma rotina que roda a cada minuto.
- Frete fixo de R$ 19,90, grátis a partir de R$ 299 de subtotal.
- O cupom é validado de novo na finalização: precisa estar ativo, dentro do período, abaixo do limite de usos e acima do pedido mínimo.
- O pedido guarda uma cópia do endereço e do nome e preço de cada item; mudar um produto depois não altera pedidos antigos.
- Os números de pedido seguem o formato **VL** seguido de seis dígitos.

Pagamento simulado

É o modo padrão. Nenhum valor é cobrado. Use qualquer nome, validade futura e CVV de 3 dígitos:

Cartão	Resultado
`4242 4242 4242 4242`	Aprovado
`5555 5555 5555 4444`	Aprovado

Cartão	Resultado
`4000 0000 0000 0002`	Recusado pelo emissor
`4000 0000 0000 9995`	Saldo insuficiente

PIX gera um código copia e cola válido por 30 minutos; o boleto gera a linha digitável com vencimento em 3 dias. Os dois deixam o pedido aguardando, e o botão **Simular pagamento** no detalhe do pedido confirma como se o banco tivesse avisado. O número do cartão nunca é gravado: ficam só a bandeira e os 4 últimos dígitos.

Stripe

1. Crie uma conta em stripe.com e copie a chave secreta de teste (`sk_test_...`).
2. Em Developers, Webhooks, cadastre `https://SEU-ENDERECO/api/payments/webhook/stripe` com os eventos `checkout.session.completed`, `checkout.session.async_payment_succeeded`, `checkout.session.async_payment_failed` e `checkout.session.expired`. Copie o segredo de assinatura (`whsec_...`).
3. Defina as variáveis `VELO_PAYMENT_PROVIDER=STRIPE`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET` e `VELO_PUBLIC_URL` e suba a aplicação.
4. Para testar localmente, use o Stripe CLI: `stripe listen --forward-to localhost:8080/api/payments/webhook/stripe`.

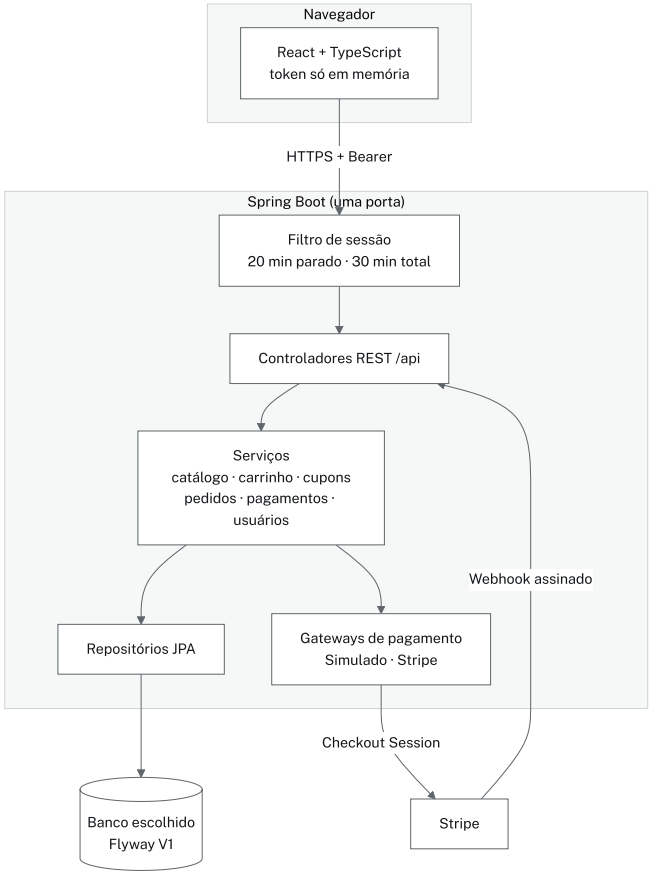
O checkout passa a levar o cliente para a página segura da Stripe. A confirmação chega pelo webhook, que tem a assinatura conferida (com tolerância de 5 minutos) antes de qualquer alteração no pedido. Ao voltar da Stripe, o cliente entra de novo, porque a página recarrega.

> **Nunca coloque chaves reais no código**
> Use variáveis de ambiente. O arquivo `.env.example` lista todas as que a Velo entende.

Capítulo 7

Arquitetura

A Velo é uma aplicação única: o Spring Boot serve a API em `/api` e a interface compilada na mesma porta.



Visão geral da arquitetura

Backend por pacote

Pacote	Responsabilidade
`auth`	Login, cadastro, sessões, filtro de autenticação e política de senha
`user`	Perfil, endereços e gestão de usuários pelo administrador
`catalog`	Categorias, produtos, busca, estoque e movimentos
`cart`	Carrinho de cada usuário
`coupon`	Cadastro, validação, uso e devolução de cupons
`order`	Checkout, pedidos, preços, ciclo de situações e cancelamento automático
`payment`	Gateways simulado e Stripe, pagamentos, estornos e webhook
`dashboard`	Indicadores e gráficos do painel

Pacote	Responsabilidade
`seed`	Dados de demonstração na primeira execução
`config`	Segurança, propriedades, relógio e entrega da interface
`common`	Erros padronizados, paginação, dinheiro, textos e datas

Interface por pasta

Pasta	Conteúdo
`auth/`	Contexto de sessão, guardas de rota e avisos de tempo
`cart/`	Contexto do carrinho, compartilhado por todas as telas
`layouts/`	Estrutura da loja, do painel, gaveta do carrinho e menu do usuário
`pages/`	Entrada, vitrine, produto, carrinho, checkout, pedidos e conta
`pages/admin/`	Visão geral, pedidos, produtos, categorias, cupons e usuários
`features/`	Formulários de endereço e pagamento, blocos do pedido e gráficos
`components/`	Logo, ícones, modais, avisos e elementos básicos
`lib/`	Cliente da API, tipos, formatação e ganchos de dados

Decisões principais

- **Sessão opaca, não JWT.** O token é aleatório e o servidor guarda só o hash; assim dá para encerrar qualquer sessão na hora e aplicar os dois limites de tempo.
- **Token só em memória.** Nada fica no armazenamento do navegador, então recarregar a página desloga, como pedido.
- **Trava pessimista no estoque.** Dois clientes comprando a última unidade ao mesmo tempo não conseguem vender o mesmo item duas vezes.
- **Versão nas tabelas.** Produto, cupom e pedido têm coluna de versão para impedir que uma edição sobrescreva outra.
- **Esquema gerado.** Um script Python é a fonte única do modelo e gera o SQL de cada banco e a documentação.
- **Relógio injetável.** Toda regra de tempo usa um relógio do Spring, o que permite testar os limites de sessão sem esperar.

Erros

Toda resposta de erro tem o mesmo formato, e a interface mostra a mensagem e destaca os campos:

```
{ "code": "VALIDATION", "message": "Revise os campos destacados.",
  "fields": { "zipCode": "CEP inválido." } }
```

Capítulo 8

API

Todas as rotas ficam em `/api`, respondem JSON e exigem o cabeçalho `Authorization: Bearer` com o token, exceto login, cadastro, saúde e o webhook.

Método	Rota	Acesso	Para que serve
POST	`/api/auth/login`	público	Entrar e receber o token
POST	`/api/auth/register`	público	Criar conta de cliente
GET	`/api/auth/me`	logado	Dados de quem está logado
POST	`/api/auth/ping`	logado	Manter a sessão ativa
POST	`/api/auth/logout`	público	Encerrar a sessão
GET	`/api/health`	público	Saúde da aplicação
GET	`/api/catalog/categories`	logado	Categorias visíveis
GET	`/api/catalog/products`	logado	Busca de produtos com filtros e paginação
GET	`/api/catalog/products/{slug}`	logado	Detalhe do produto
GET	`/api/cart`	logado	Carrinho com totais (aceita ?coupon=)
POST	`/api/cart/items`	logado	Adicionar item
PUT	`/api/cart/items/{productId}`	logado	Mudar quantidade
DELETE	`/api/cart/items/{productId}`	logado	Remover item
DELETE	`/api/cart`	logado	Esvaziar o carrinho
POST	`/api/orders`	logado	Finalizar a compra
GET	`/api/orders`	logado	Meus pedidos
GET	`/api/orders/{número}`	logado	Detalhe do meu pedido
POST	`/api/orders/{número}/payments`	logado	Pagar ou tentar de novo
POST	`/api/orders/{número}/cancel`	logado	Cancelar meu pedido
GET	`/api/payments/options`	logado	Formas de pagamento e cartões de teste
POST	`/api/payments/{id}/simulate`	logado	Confirmar ou recusar PIX/boleto simulado
POST	`/api/payments/webhook/stripe`	Stripe	Avisos de pagamento da Stripe (assinados)
GET, PUT	`/api/account`	logado	Ver e editar dados pessoais
PUT	`/api/account/password`	logado	Trocar a senha
GET, POST	`/api/account/addresses`	logado	Listar e criar endereços

Método	Rota	Acesso	Para que serve
PUT, DELETE	`/api/account/addresses/{id}`	logado	Editar e excluir endereço
GET	`/api/admin/dashboard`	equipe	Indicadores (aceita ?days=)
GET	`/api/admin/orders`	equipe	Pedidos com busca e filtros
GET	`/api/admin/orders/{número}`	equipe	Detalhe do pedido
PUT	`/api/admin/orders/{número}/status`	equipe	Mudar situação e rastreio
GET, POST	`/api/admin/products`	equipe	Listar e criar produtos
GET, PUT, DELETE	`/api/admin/products/{id}`	equipe	Ver, editar e excluir produto
POST	`/api/admin/products/{id}/stock`	equipe	Movimentar estoque
GET	`/api/admin/products/{id}/movements`	equipe	Histórico de estoque
GET, POST	`/api/admin/categories`	equipe	Listar e criar categorias
PUT, DELETE	`/api/admin/categories/{id}`	equipe	Editar e excluir categoria
GET, POST	`/api/admin/coupons`	equipe	Listar e criar cupons
PUT, DELETE	`/api/admin/coupons/{id}`	equipe	Editar e excluir cupom
GET, POST	`/api/admin/users`	admin	Listar e criar usuários
GET, PUT	`/api/admin/users/{id}`	admin	Ver e editar usuário
PUT	`/api/admin/users/{id}/password`	admin	Redefinir senha (encerra as sessões)
POST	`/api/admin/users/{id}/sessions/revoke`	admin	Encerrar sessões abertas

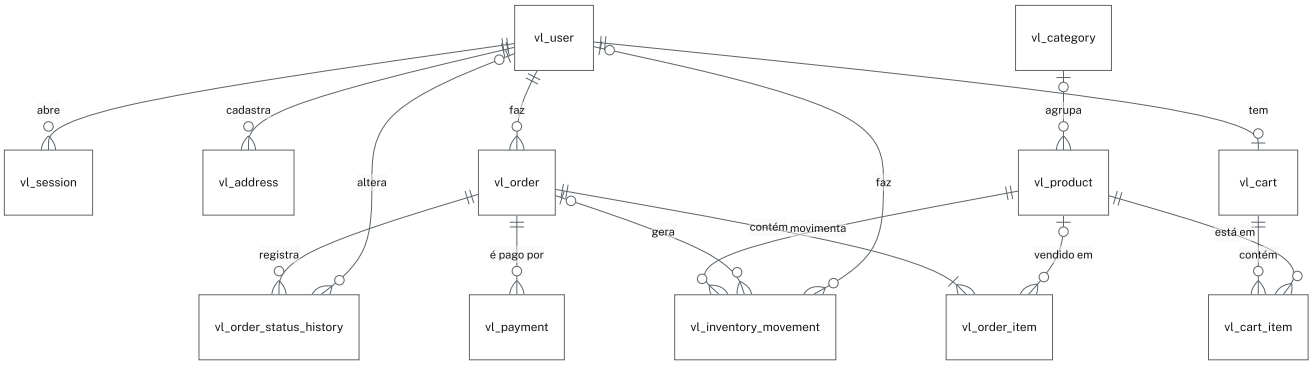
Respostas de sessão trazem os cabeçalhos `X-Velo-Idle-Expires-At` e `X-Velo-Expires-At`, usados pela interface para mostrar o tempo restante.

Capítulo 9

Banco de dados

A Velo funciona com o banco que você preferir. As 13 tabelas são criadas sozinhas na primeira execução, com o Flyway.

O guia completo, com o diagrama de todos os campos e chaves, está no PDF **Banco de dados da Velo** (na raiz do projeto e em `docs/`) e em `docs/banco-de-dados.md`. Aqui fica o resumo.



Relacionamentos entre as tabelas

Pareando com outro banco

1. Suba o banco. Com Docker: `docker compose --profile postgresql up -d` (ou mysql, mariadb, sqlserver, oracle).
2. Sem Docker, crie o banco e o usuário `velo` com o SQL do guia.
3. Escolha o perfil: `SPRING_PROFILES_ACTIVE=postgresql` e, se o banco não estiver no endereço padrão, `VELO_DB_URL`, `VELO_DB_USER` e `VELO_DB_PASSWORD`.
4. Rode a aplicação. O Flyway cria as tabelas e a Velo cadastra os dados de demonstração.

Perfil	URL padrão	Usuário / senha
`(nenhum)`	`jdbc:h2:file:./data/velo`	sa / (vazia)
`postgresql`	`jdbc:postgresql://localhost:5432/velo`	velo / velo
`mysql`	`jdbc:mysql://localhost:3306/velo`	velo / velo
`mariadb`	`jdbc:mariadb://localhost:3307/velo`	velo / velo
`sqlserver`	`jdbc:sqlserver://localhost:1433;databaseName=velo`	velo / Velo@Senha2026
`oracle`	`jdbc:oracle:thin:@//localhost:1521/FREEPDB1`	velo / velo

Capítulo 10

Configuração, testes e automação

Variáveis de ambiente

Variável	Padrão	Para que serve
`SPRING_PROFILES_ACTIVE`	`(vazio, H2)`	postgresql, mysql, mariadb, sqlserver ou oracle
`VELO_DB_URL`	`depende do perfil`	Endereço JDBC do banco
`VELO_DB_USER` / `VELO_DB_PASSWORD`	`depende do perfil`	Credenciais do banco
`VELO_PORT`	`8080`	Porta do servidor
`VELO_SEED`	`true`	Cadastra os dados de demonstração quando o banco está vazio
`VELO_PUBLIC_URL`	`http://localhost:8080`	Endereço público, usado no retorno da Stripe
`VELO_PAYMENT_PROVIDER`	`SIMULATED`	SIMULATED ou STRIPE
`STRIPE_SECRET_KEY` / `STRIPE_WEBHOOK_SECRET`	`(vazio)`	Chaves da Stripe

Outras regras ficam em `application.yml`: limites de sessão, prazo de pagamento, frete, valor para frete grátis e número máximo de parcelas.

Testes

```
cd backend
./mvnw -B verify
```

Teste	O que confere
`SessionLimitsTest`	Encerramento por 20 minutos parado e por 30 minutos de vida, com relógio controlado
`CheckoutFlowTest`	Compra com cartão aprovado e recusado, PIX, cupom, estoque, rastreio e cancelamento
`HttpApiTest`	Respostas 401 e 403, permissões de cada perfil, painel, cadastro e saída
`UnitTests`	Validação de cartão (Luhn), textos, política de senha e assinatura da Stripe

Integração contínua

A cada envio para a branch main, o GitHub Actions compila a interface e roda os testes do backend (`.github/workflows/ci.yml`).

Commits automáticos

O workflow `.github/workflows/diario.yml` registra uma linha em `.github/atividade.txt` e faz um commit todos os dias às 17h e às 23h (horário de Brasília), mantendo o gráfico de contribuições do GitHub ativo. Ele usa os segredos `GIT_NAME` e `GIT_EMAIL` do repositório; o passo a passo para ativar está no README.

Capítulo 11

Solução de problemas

Sintoma	O que fazer
java não é reconhecido	Instale o Java 21 e abra um novo terminal. Confira com `java -version`.
A porta 8080 já está em uso	Feche o outro programa ou defina `VELO_PORT=8081`.
Fui deslogado sozinho	É o esperado ao recarregar a página, depois de 20 minutos parado ou 30 minutos de sessão. Entre de novo.
E-mail bloqueado por alguns minutos	Houve 5 tentativas erradas. Espere 5 minutos ou peça ao administrador para redefinir a senha.
Connection refused ao iniciar	O banco escolhido não está rodando ou a porta é outra. Confira com `docker compose ps` ou ajuste `VELO_DB_URL`.
Migrations have failed validation	O banco já tem outro esquema. Use um banco vazio só para a Velo.
Quero começar do zero no H2	Pare a aplicação e apague a pasta `backend/data`.
A interface não aparece em localhost:8080	Rode sem `-DskipUi` para o Maven compilar e embutir a interface.
Pagamento Stripe não confirma	Confira o webhook, o segredo `whsec_` e se `VELO_PUBLIC_URL` aponta para um endereço acessível pela Stripe.